

K L Deemed to be University

Department of Computer Science and Engineering | 2026-2027, Odd Semester
Database Systems Engineering and Distributed Backend Development (25CS1302E)

Practical Experiment 7: User Profiles and Activity Logging

Name	Habeeb	Roll No.	2520090054
Course	25CS1302E	CO Mapping	CO4
Session	Practical 7	Date	

Aim

To build an independent Node.js user-service (Express + Mongoose) on port 5001 that stores user profiles with a flexible profile object and activity logs in MongoDB, with POST /users, GET /users/:id, POST /activities and GET /activities/:user_id.

Program

Setup: mkdir user-service && cd user-service && npm init -y && npm install express mongoose

db.js

```
// db.js - MongoDB connection and schemas (blueprints)
const mongoose = require("mongoose");

const MONGO_URI = process.env.MONGO_URI || "mongodb://127.0.0.1:27017/bookflow_users";

async function connectDB() {
  try {
    await mongoose.connect(MONGO_URI);
    console.log(`MongoDB connected: ${mongoose.connection.name}`);
  } catch (err) {
    console.error("MongoDB connection failed:", err.message);
    process.exit(1);
  }
}

// User: fixed fields plus a flexible profile object (any shape per user)
const userSchema = new mongoose.Schema(
  {
    username: { type: String, required: true, unique: true, trim: true },
    email: {
      type: String,
      required: true,
      unique: true,
      lowercase: true,
      match: [/^\S+@\S+\.\S+$/, "Invalid email address"],
    },
    profile: { type: mongoose.Schema.Types.Mixed, default: {} },
  },
  { versionKey: false }
);

// Activity: one log entry per user action
const activitySchema = new mongoose.Schema(
  {
    user_id: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true, index: true },
    action: {
      type: String,
      required: true,
      enum: ["LOGIN", "LOGOUT", "VIEW_BOOK", "SEARCH", "BORROW_BOOK", "RETURN_BOOK"],
    },
    details: { type: String },
    timestamp: { type: Date, default: Date.now },
  },
  { versionKey: false }
);

const User = mongoose.model("User", userSchema);
const Activity = mongoose.model("Activity", activitySchema);

module.exports = { connectDB, User, Activity };
```

routes.js

```
// routes.js - user profile and activity log endpoints
const express = require("express");
const mongoose = require("mongoose");
const { User, Activity } = require("../db");

const router = express.Router();

// Wrap async handlers so rejected promises reach the error middleware
const asyncHandler = (fn) => (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next);

const isValidId = (id) => mongoose.Types.ObjectId.isValid(id);

// POST /users - add a new user profile
router.post(
  "/users",
  asyncHandler(async (req, res) => {
    const { username, email, profile } = req.body;
    const user = await User.create({ username, email, profile });
    res.status(201).json(user);
  })
);

// GET /users/:id - find a specific user by ID
router.get(
  "/users/:id",
  asyncHandler(async (req, res) => {
    if (!isValidId(req.params.id)) {
      return res.status(400).json({ error: "Invalid user id" });
    }
    const user = await User.findById(req.params.id);
    if (!user) {
      return res.status(404).json({ error: "User not found" });
    }
    res.json(user);
  })
);

// POST /activities - save a new activity log
router.post(
  "/activities",
  asyncHandler(async (req, res) => {
    const { user_id, action, details } = req.body;
    if (!isValidId(user_id)) {
      return res.status(400).json({ error: "Invalid user_id" });
    }
    const userExists = await User.exists({ _id: user_id });
    if (!userExists) {
      return res.status(404).json({ error: "User not found" });
    }
    const activity = await Activity.create({ user_id, action, details });
    res.status(201).json(activity);
  })
);

// GET /activities/:user_id - list all past actions of a user (newest first)
router.get(
  "/activities/:user_id",
  asyncHandler(async (req, res) => {
    if (!isValidId(req.params.user_id)) {
      return res.status(400).json({ error: "Invalid user_id" });
    }
    const activities = await Activity.find({ user_id: req.params.user_id }).sort({ timestamp: -1 });
    res.json({ user_id: req.params.user_id, count: activities.length, activities });
  })
);

module.exports = router;
```

server.js

```
// server.js - Express entry point for the user-service (port 5001)
const express = require("express");
const { connectDB } = require("../db");
const routes = require("../routes");

const PORT = process.env.PORT || 5001;
const app = express();
```

```

app.use(express.json()); // parse JSON request bodies

app.get("/health", (req, res) => res.json({ service: "user-service", status: "UP" }));
app.use("/", routes);

// 404 for unknown routes
app.use((req, res) => res.status(404).json({ error: `Route ${req.method} ${req.path} not found` }));

// Central error handler: validation errors -> 400, duplicates -> 409, others -> 500
app.use((err, req, res, next) => {
  if (err.name === "ValidationError") {
    const messages = Object.values(err.errors).map((e) => e.message);
    return res.status(400).json({ error: "Validation failed", details: messages });
  }
  if (err.code === 11000) {
    return res.status(409).json({ error: "Duplicate value: username or email already exists" });
  }
  console.error(err);
  res.status(500).json({ error: "Internal server error" });
});

connectDB().then(() => {
  app.listen(PORT, () => console.log(`user-service running on http://localhost:${PORT}`));
});

```

test_p7.sh

```

#!/bin/bash
# Experiment 7 - user-service tests (run after: node server.js)
BASE=http://localhost:5001
call() {
  echo "\$ curl \$*"
  body=$(curl -s -w '\n%{http_code}' "$@" )
  code=$(echo "$body" | tail -n1); json=$(echo "$body" | sed '$d')
  echo "HTTP $code"
  [ -n "$json" ] && echo "$json" | python3 -m json.tool --indent 2
  echo
}
echo "### 1. POST /users - user with a bio"
call -X POST $BASE/users -H 'Content-Type: application/json' \
  -d '{"username":"habeeb","email":"habeeb@kluniversity.in","profile":{"bio":"CSE student who loves
    sci-fi","favourite_genres":["Science Fiction","Computer Science"]}}'
U1=$(curl -s -X POST $BASE/users -H 'Content-Type: application/json' \
  -d '{"username":"ananya","email":"ananya.rao@gmail.com","profile":{"social_links":{"github":"github.com/ananya
    rao","linkedin":"linkedin.com/in/ananyarao"},"year":2}}' | python3 -c 'import sys,json;print(json.load(sys.
    stdin)["_id"])')
echo "### 2. POST /users - a second user with a different profile shape (social links)"
echo "(created user _id = $U1)"
echo
echo "### 3. GET /users/:id"
call $BASE/users/$U1
echo "### 4. POST /users - duplicate username is rejected"
call -X POST $BASE/users -H 'Content-Type: application/json' -d '{"username":"ananya","email":"other@gmail.com"}'
echo
echo "### 5. POST /users - invalid email is rejected"
call -X POST $BASE/users -H 'Content-Type: application/json' -d '{"username":"rahul","email":"rahul-at-mail"}'
echo "### 6. POST /activities - log three actions for the user"
call -X POST $BASE/activities -H 'Content-Type: application/json' -d '{"user_id":"$U1","action":"LOGIN",
  "details":{"User logged in"}}'
sleep 1
curl -s -o /dev/null -X POST $BASE/activities -H 'Content-Type: application/json' -d
  '{"user_id":"$U1","action":"SEARCH","details":{"Searched for 'Python'}}'
sleep 1
curl -s -o /dev/null -X POST $BASE/activities -H 'Content-Type: application/json' -d
  '{"user_id":"$U1","action":"VIEW_BOOK","details":{"Viewed book_id 3 (The Martian)}}'
echo "### 7. GET /activities/:user_id - all past actions (newest first)"
call $BASE/activities/$U1
echo "### 8. GET /users/:id for an unknown user"
call $BASE/users/6650a1b2c3d4e5f601234567

```

Output

node server.js

```

MongoDB connected: bookflow_users
user-service running on http://localhost:5001

```

bash test_p7.sh

```
### 1. POST /users - user with a bio
$ curl -X POST http://localhost:5001/users -H Content-Type: application/json -d {"username":"habeeb","email":"habeeb@kluniversity.in","profile":{"bio":"CSE student who loves sci-fi","favourite_genres":["Science Fiction","Computer Science"]}}
HTTP 201
{
  "username": "habeeb",
  "email": "habeeb@kluniversity.in",
  "profile": {
    "bio": "CSE student who loves sci-fi",
    "favourite_genres": [
      "Science Fiction",
      "Computer Science"
    ]
  },
  "_id": "6abf9f3d5ee33302ff79e3e7"
}

### 2. POST /users - a second user with a different profile shape (social links)
(created user _id = 6abf9f3d5ee33302ff79e3e8)

### 3. GET /users/:id
$ curl http://localhost:5001/users/6abf9f3d5ee33302ff79e3e8
HTTP 200
{
  "_id": "6abf9f3d5ee33302ff79e3e8",
  "username": "ananya",
  "email": "ananya.rao@gmail.com",
  "profile": {
    "social_links": {
      "github": "github.com/ananyarao",
      "linkedin": "linkedin.com/in/ananyarao"
    },
    "year": 2
  }
}

### 4. POST /users - duplicate username is rejected
$ curl -X POST http://localhost:5001/users -H Content-Type: application/json -d {"username":"ananya","email":"otter@gmail.com"}
HTTP 409
{
  "error": "Duplicate value: username or email already exists"
}

### 5. POST /users - invalid email is rejected
$ curl -X POST http://localhost:5001/users -H Content-Type: application/json -d {"username":"rahul","email":"rahul-at-mail"}
HTTP 400
{
  "error": "Validation failed",
  "details": [
    "Invalid email address"
  ]
}

### 6. POST /activities - log three actions for the user
$ curl -X POST http://localhost:5001/activities -H Content-Type: application/json -d {"user_id":"6abf9f3d5ee33302ff79e3e8","action":"LOGIN","details":"User logged in"}
HTTP 201
{
  "user_id": "6abf9f3d5ee33302ff79e3e8",
  "action": "LOGIN",
  "details": "User logged in",
  "_id": "6abf9f3e5ee33302ff79e3eb",
  "timestamp": "2026-10-02T12:10:38.139Z"
}

### 7. GET /activities/:user_id - all past actions (newest first)
$ curl http://localhost:5001/activities/6abf9f3d5ee33302ff79e3e8
HTTP 200
{
  "user_id": "6abf9f3d5ee33302ff79e3e8",
  "count": 3,
  "activities": [
    {
      "_id": "6abf9f405ee33302ff79e3ed",
      "user_id": "6abf9f3d5ee33302ff79e3e8",

```

```
"action": "VIEW_BOOK",
"details": "Viewed book_id 3 (The Martian)",
"timestamp": "2026-10-02T12:10:40.226Z"
},
{
  "_id": "6abf9f3f5ee33302ff79e3ec",
  "user_id": "6abf9f3d5ee33302ff79e3e8",
  "action": "SEARCH",
  "details": "Searched for 'Python'",
  "timestamp": "2026-10-02T12:10:39.207Z"
},
{
  "_id": "6abf9f3e5ee33302ff79e3eb",
  "user_id": "6abf9f3d5ee33302ff79e3e8",
  "action": "LOGIN",
  "details": "User logged in",
  "timestamp": "2026-10-02T12:10:38.139Z"
}
]
}

### 8. GET /users/:id for an unknown user
$ curl http://localhost:5001/users/6650a1b2c3d4e5f601234567
HTTP 404
{
  "error": "User not found"
}
```

Result

The service connected to MongoDB and listened on port 5001. Two users with differently shaped profile objects (bio and genres versus social links) were stored in the same collection, GET /users/:id returned the profile, duplicates (409) and invalid emails (400) were rejected, and three activities were logged and listed newest first. The service runs completely separately from the Python FastAPI service.